
Sudoku Solver

Author:

Corey PICKETT

May 4, 2025

1 Introduction

The game of Sudoku has existed for centuries, with origins that can be traced back to the creation of 'Latin Squares' in the 18th century [1]. The game as we know it was created by American Howard Garns in 1979 and became popular in Japan in 1984, where it was given the now iconic name 'Sudoku'.

Sudoku is a logic-based, combinatorial number-placement puzzle [2]. The objective of the game is to fill a 9 x 9 grid with the digits 1 to 9. The difficulty of the puzzle comes with ensuring that each row, column, 3 x 3 sub grid contains each digit only once. Sudoku is a common puzzle game seen in newspapers and mobile apps alike. Clues are given to the player in the form of pre-filled numbers, with the difficulty of each puzzle adjusted by the number of available clues. Figure 1 demonstrates a partially filled and solved Sudoku puzzles side by side.

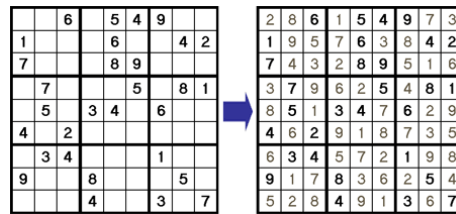


Figure 1: A solved Sudoku puzzle

Solving a Sudoku puzzle can be classified as a constraint satisfaction problem [3]. The challenge of solving such a problem lies in the ability of a model to learn and predict a correct solution effectively for unseen puzzles.

The construction of a Sudoku solving model primarily engages with the course topics of 'Automated Reasoning and Logic', as well as 'Artificial Neural Networks'. As this puzzle is a constraint satisfaction problem, the logic and reasoning given to the model prior to training is very important. Additionally, a CNN (Convolutional Neural Network) model [4] trained on pre-generated Sudoku puzzles with validation puzzles and final examples.

2 Methodology

The chosen approach for this assignment was a CNN model (See Figure 2). A CNN model was chosen for the approach due to its ability to automatically learn relevant features of a dataset, without the need for manual feature engineering. CNN's also generally perform with high accuracy and are typically robust to noise within a given dataset, allowing them to perform with higher-effectiveness to the chosen material [5]. CNN is an interesting choice of model as rather than learn with traditional rules of the game supplied, it will have to generalise with its predictions.

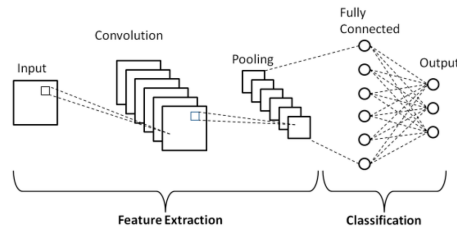


Figure 2: A simple CNN architecture diagram

This model was trained upon a dataset of generated Sudoku puzzles separated into a main test set and validation set of smaller size. As the puzzles are generated each time the program is run, the puzzles created will always be random and vary between creations. This can be especially useful for multiple runs of the program, however, over a large dataset this would be less of a benefit. Using print statements, I gathered information during the training process such as Loss and accuracy of the model averaged for both of the test and validation sets. The model was then tested on a one-off example of a Sudoku puzzle similar to the training and validation sets, where accuracy is also measured.

One-Hot encoding was used in order to make the process of learning easier for the CNN model. The model can learn quicker as it doesn't have to learn treat digits as continuous values. Instead, ten channels were implemented, representing zero as an empty cell and digits one through nine. In addition, a cross entropy loss function was implemented as each cell has nine possible classes, defining the task as a multi-class classification problem. A cross entropy function would encourage the model to output a probability

distribution, predicting the digit with the highest probability.

3 Implementation

The software was developed incorporating a CNN to predict missing digits in a partially filled Sudoku puzzle. Although Sudoku is a constraint satisfaction problem, the designed model treats the task as a series of multi-class classification problems, as it attempts to predict the most likely digit out of 1 - 9 for each empty cell.

The model takes a 9 x 9 grid as input which were hot-encoded for ease of learning, and memory saving. The model was then trained on randomly generated valid Sudoku puzzles and their subsequent solutions. Training was continuously monitored with regular updates on loss and accuracy for each epoch. 'Clue' cells were not included in these accuracy reports, as they were already filled prior to the model predicting a solution. The CNN model outputs a 9 x 9 x 9 tensor logits, which is then processed with the `argmax()` function to predict final values of puzzles. Following training, the model with the highest accuracy score during validation is selected for further evaluation, involving 5 new puzzles with accuracy recorded, and then one final puzzle printed with the original puzzle, the predicted solution, and the actual solution for comparison.

A variety of tools and libraries were utilised in the development of this software. The software itself was created using the programming language 'Python' [6]. A number of libraries were utilised for various applications within the software, these include; Pytorch [7], NumPy [8], Random, and Copy. Pytorch was primarily used for machine learning, whilst NumPy was used for handling the matrices created during runtime. The modules random and copy were used to shuffle cell positions within puzzles and copy original puzzles to avoid modifying objects respectively.

There were a variety of challenges to overcome during the process of development. In order to represent Sudoku in a way that preserves the constraints of the puzzles required very careful one-hot encoding. This was something

I originally struggled with, however with conjunction with Gen AI such as Chat-GPT I was able to overcome this issue and preserve the sparsity and digit identity required for the model. It is also important to mention Gen AI was consulted if I encountered an error of which I couldn't diagnose successfully alone, which was a great time saver over the course of development. In order to mitigate over fitting, especially for a small dataset, batch normalisation and dropout were used through the training process. Finally, as a CNN model doesn't enforce the constraints of a Sudoku, invalid inputs could sometimes occur, however this is mitigated with a large training and validation dataset, giving the model more to learn from.

4 Results and Discussion

For results discussed in this section the following parameters were enabled:

- 30 Epochs
- Dataset size of 100,000
- Batch size of 64

The results of this experiment claimed a final evaluation accuracy of 75 percent, with 30 / 40 cells correctly predicted by the model. This can be further validated by the 5 puzzle evaluation which generated an average accuracy of 67 per cent. The time taken for the model to train and be evaluated was approx. 1 hour, however this may have been more due to computational restraints of my machine, rather than the complexity of the model designed.

The average training loss (Figure 3) decreased steadily over the course of most of the training time, beginning to plateau much more towards the end of the training cycle, indicating the model may not be able to improve much more beyond its current ability without tweaks to parameters. Similarly, when model accuracy (Figure 4) is plotted across iterations we can see a steady increase with a fall off towards the end of the training cycle. Validation loss (Figure 5) and accuracy (Figure 6) saw less drastic changes across the training cycle, with both metrics improving much less then compared to their training counterparts. This trend is likely down to the much



Figure 3: Training Loss

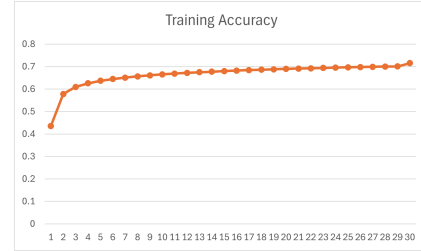


Figure 4: Training Accuracy

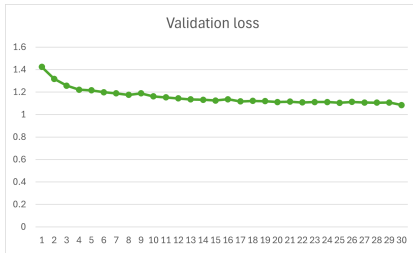


Figure 5: Validation Loss

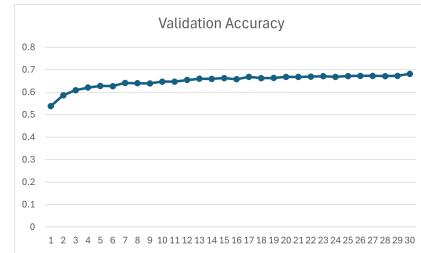


Figure 6: Validation Accuracy

smaller size of the validation dataset, as only 500 puzzles were selected for the validation phase.

Overall the model was somewhat effective and not very suited to solving Sudoku puzzles. It clearly has the ability to successfully predict the correct digits in a partially filled Sudoku puzzle, however, it would require a large dataset and could be computationally expensive. The model may also struggle to handle puzzles with very few clues (less than 30) when compared to the 30-40 used for this experiment. When comparing my results to other benchmarks such as a backtracking solver, the designed model is not nearly as effective.

5 Conclusion and Future Work

Following the completion of the CNN model and testing, it can be concluded that a CNN model can be effective at approximating a Sudoku puzzle solution, given a sufficient dataset. However, when compared to alternative techniques, the model is far from first choice. By training on large gener-

ated datasets, the model learns to generalise number-placement strategies without requiring the constraints of Sudoku itself. This can lead to invalid digit placement in some instances. Due to this large data need, the model may suffer decreased performance when dealing with puzzles with few clues, especially as there is currently no constraint enforcement in the software.

Ethical considerations when implementing this software could involve misuse issues. Due to no guarantee of correctness, if the model is used in a high-stakes domain such as a medical diagnosis, problems can arise. A risk of using this model are potential bias in training data. This could occur if the parameters of the training puzzles are modified and no longer reflect real-world examples, leading to poor performance. Finally, this model suffers from limitations such as a large dependence on dataset size, and potential lack of validity.

There are a variety of future improvements or extensions that can be applied to the developed model. Combining the model's prediction with a constraint solver to enforce Sudoku rules would likely lead to much higher accuracy, at the cost of time to compute. Also, training the model on a larger dataset of varied puzzles would be likely to improve prediction accuracy for the model. The model could be extended to other grid-based solvers, such as KenKen [9] or Kakuro [10].

References

- [1] History of sudoku.
URL <https://sudoku.com/how-to-play/the-history-of-sudoku/>
- [2] How to play sudoku.
URL <https://sudoku.com/how-to-play/sudoku-rules-for-complete-beginners/>
- [3] H. Simonis, Sudoku as a constraint problem, in: CP Workshop on modeling and reformulating Constraint Satisfaction Problems, Vol. 12, Cite-seer Sitges, Spain, 2005, pp. 13–27.

- [4] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. Al-Amidie, L. Farhan, Review of deep learning: Concepts, cnn architectures, challenges, applications, future directions, *Journal of Big Data* 8 (1). doi:10.1186/s40537-021-00444-8.
- [5] S. Jana, M. Sen, A. Khan, A. K. Maji, R. K. Pal, Analysing the effectiveness of different machine learning approaches towards solving sudoku puzzle, *Lecture Notes in Networks and Systems* (2024) 25–41doi:10.1007/978-981-97-8476-9₃.
- [6] Python.
URL <https://www.python.org/>
- [7] Pytorch.
URL <https://pytorch.org/>
- [8] Numpy.
URL <https://numpy.org/>
- [9] Kenken (Apr 2025).
URL <https://en.wikipedia.org/wiki/KenKen>
- [10] Kakuro (Feb 2025).
URL <https://en.wikipedia.org/wiki/Kakuro>